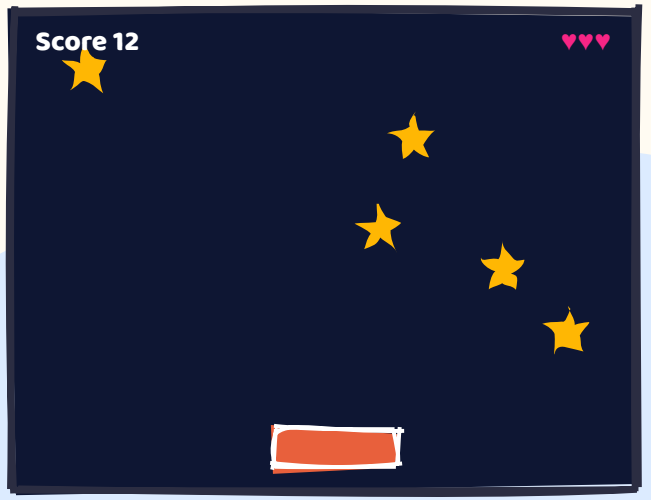
Young Innovators · Class 19

Python Pygame: Catch the Falling Stars

Your first real game, in real Python

Game loop | Collisions | Lives & levels



Did you know?
Python is used by game studios for tools and scripting, and Pygame has powered thousands of indie games.

LEARN

Let's think...

What happens 60 times a second?

Frame by frame

A game looks smooth, but it is really a **flip-book**: 60 still pictures every second. Between each picture the program reads your keys, moves everything, and checks for crashes. What would happen if it skipped one step?



Nothing?

It freezes!

Glitches!

Today: you write **Catch the Falling Stars** in Python with Pygame: a basket, falling stars, score, lives and game over.

Learning objectives

By the end of this class I can...

1	**Game loop:** explain the **game loop**: handle input → update → draw, 60 times a second	**Proof:** I can point to the 3 parts in my code
2	**Move and collide:** move a player with the keyboard and detect **collisions** with **Rect**s	**Proof:** my basket catches stars and the score goes up
3	**Make it a game:** add **lives**, rising **difficulty** and a **game over** screen	**Proof:** a friend played my game and lost on purpose... or not!

LEARN

Objective 3 · Make it a game · video

Video time!



2 YEARS of PYTHON Game Development in 5 Minutes!

Coding With Russ · about 5 minutes · youtu.be/lzHLPaU7UUE

Watch for...

1. What kinds of games did he build with **Python + Pygame**?
2. What did he find hard at the **start**?
3. Which feature made his games feel **fun**?

Scan to watch

After the video: Your first Pygame game is today. Which feature from the video would you add to it?

LEARN

Objective 1 · Game loop

The game loop

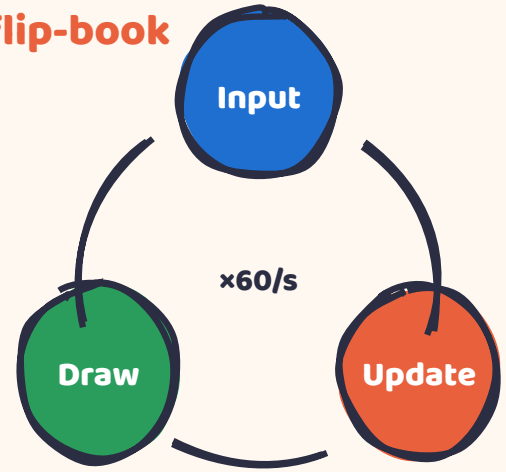
The idea

Every game runs the same loop, again and again:

1. **Input**: which keys are pressed? Did the player close the window?
2. **Update**: move the basket, move the stars, check collisions, change the score.
3. **Draw**: clear the screen, draw everything, flip it to the display.

clock.tick(60) keeps it at 60 frames per second.

Example: a flip-book



Draw a stick figure on the corner of each page, moving a little each time. Flip fast: it walks! Each page is one frame of the loop.

Objective 2 · Move and collide

Position, speed and Rects

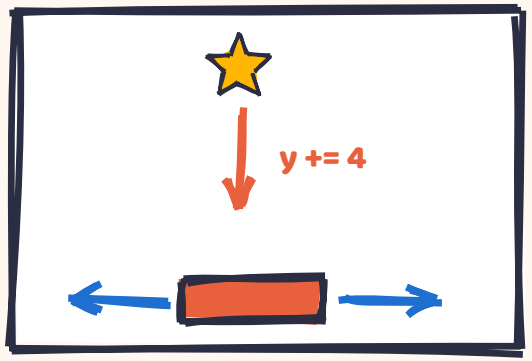
The idea

Everything on screen is a **Rect**: x, y, width, height. Remember: y grows **downwards**.

Speed = pixels moved per frame. A star with speed 4 does **star.y += 4** each frame → 240 px per second.

Keys: **if keys[pygame.K_LEFT]: basket.x -= 7**

Example: a lift in a building



A lift going down 1 floor every 2 seconds: you can predict where it will be. Stars fall at a steady speed, so the player can predict and catch them.

Objective 3 · Make it a game

Collisions, score and lives

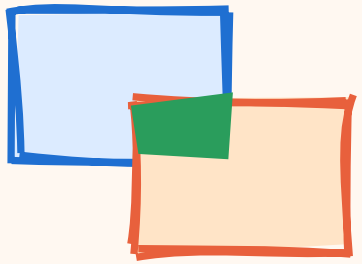
The idea

basket.colliderect(star) is True when the two rectangles overlap.

Caught → **score += 1**, move the star back to the top.
Missed (star.y > height) → **lives -= 1**.
lives == 0 → **game over**.

Make it harder over time: **speed = 3 + score // 5**.

Example: catching a cricket ball



overlap = catch!

A fielder's hands and the ball must be in the same place at the same time. The game just checks: do the two boxes overlap this frame?

Real world

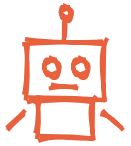
Game loops everywhere



Every video game
From Snake to Minecraft: input → update → draw



Driving simulators
Run physics hundreds of times a second



Robots
Sense → think → act is a game loop too



Phone apps
Redraw the screen every frame you scroll

? Which game do you play that has 'lives' and gets harder? How does it get harder?

Robots too!
Your Arduino loop() is the same idea: read sensors, decide, act, repeat.

Quick quiz

Game logic

Question	Answer
1. Name the 3 steps of the game loop.	
2. Speed 5, 60 FPS. Pixels per second?	
3. A star at y = 0 falls 4 px per frame. y after 10 frames?	
4. Star.y > height means...?	
5. score = 12. speed = 3 + score // 5 = ?	

Answers

Cover! 1: input, update, draw · 2: 300 · 3: 40 · 4: missed, lose a life · 5: 5

Part 2 · Make it

Now let's put it to work!

You know the ideas. Time to play, build and test. Use these slides while you work:

1. Human game loop
2. Set up (once)
3. Window, basket and input
4. Update and draw
5. Make it a real game
6. Playtest log
7. Game designer challenges

Class game

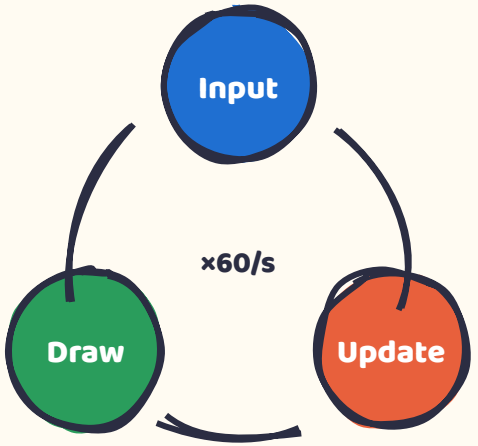
Human game loop

How to play

1. Three students are the **loop**: INPUT, UPDATE, DRAW.
2. A 'player' calls left/right. INPUT tells UPDATE. UPDATE moves a paper basket on the board.
3. DRAW shouts 'frame!' and the class claps. A 'star' magnet moves down 1 square each frame.
4. Speed up! What breaks first when the loop gets faster?

Then discuss

What happens if UPDATE forgets to check the collision before DRAW?



Big idea

Every frame, every step, in the same order.

Build it

Set up (once)

Thonny
Python editor we already use

Tools → Manage packages
Search pygame, click Install

Test
import pygame → no error = ready

Images (optional)
star.png and basket.png from the class folder

Sound (optional)
ding.wav for a catch

Save as
stars.py in your folder

No images? Start with coloured rectangles and circles. Add art at the end.

Close the window
Always handle pygame.QUIT, or the window freezes and you must stop it from Thonny (red Stop button).

Objective 1 · Game loop · Build step 1 of 3

Window, basket and input

```
import pygame, random
pygame.init()
W, H = 600, 700
screen = pygame.display.set_mode((W, H))
pygame.display.set_caption("Catch the Falling Stars")
clock = pygame.time.Clock()
font = pygame.font.SysFont(None, 40)

basket = pygame.Rect(W//2 - 50, H - 50, 100, 20)
star = pygame.Rect(random.randint(0, W - 30), 0, 30, 30)
score, lives, running = 0, 3, True

while running:
    # 1. INPUT
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False
    keys = pygame.key.get_pressed()
    if keys[pygame.K_LEFT] and basket.left > 0:  basket.x -= 7
    if keys[pygame.K_RIGHT] and basket.right < W: basket.x += 7
```

Rect

pygame.Rect(x, y, width, height). The basket sits 50 px above the bottom.

Not done!

This is the top half. The loop continues on the next slide.

Keys

get_pressed() lets you hold the key down for smooth movement.

MAKE

Objective 2 · Move and collide · Build step 2 of 3

Update and draw

```
    # 2. UPDATE
    speed = 3 + score // 5            # harder over time
    star.y += speed
    if basket.colliderect(star):      # caught!
        score += 1
        star.topleft = (random.randint(0, W - 30), -30)
    elif star.top > H:                # missed
        lives -= 1
        star.topleft = (random.randint(0, W - 30), -30)
    if lives == 0:
        running = False

    # 3. DRAW
    screen.fill((14, 22, 51))
    pygame.draw.rect(screen, (232, 96, 60), basket, border_radius=6)
    pygame.draw.ellipse(screen, (255, 183, 3), star)
    hud = font.render(f"Score {score}   Lives {lives}", True, (255, 255, 255))
    screen.blit(hud, (15, 10))
    pygame.display.flip()
    clock.tick(60)

print("Game over! Score:", score)
pygame.quit()
```

Indent

Everything here is **inside** the while loop: 4 spaces in.

Order

fill() first wipes the old frame, then draw, then flip() shows it.

Test

Run it! Catch 5 stars: do they fall faster?

MAKE

Objective 3 · Build step 3 of 3

Make it a real game

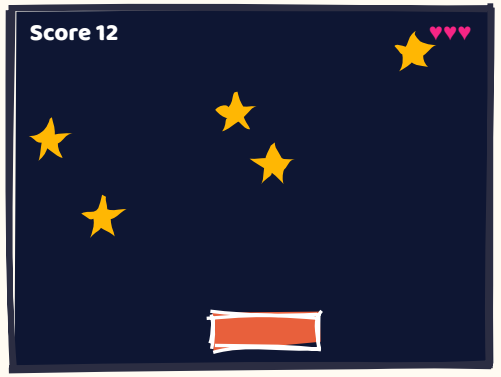
a **Many stars**: make a list of 3 Rects and loop over it in UPDATE and DRAW.

b **Game over screen**: instead of quitting, show 'GAME OVER, press R'. R resets score and lives.

c **Bad stars**: a red meteor that costs a life if you catch it.

d **Art and sound**: pygame.image.load('star.png'), pygame.mixer.Sound('ding.wav').play().

e **Playtest**: swap seats. Write one thing to improve on a sticky note.



Check: A friend can play, lose, and restart without help.

Test like an engineer

Playtest log

Test	Pass?	Fix needed
Basket stops at both edges		
Catch → score +1		
Miss → lives −1		
Gets faster every 5 points		
Friend's score / their comment		

Debug checklist

IndentationError? Line up the blocks: 4 spaces inside while, 8 inside if.
Window 'not responding'? You need the for event loop.
Star never caught? Print basket and star to check their positions.

Good games are easy to learn and hard to master. Is yours?

MAKE

Level up!

Game designer challenges

Choose your challenge



High score: save the best score to a file with open('best.txt', 'w').



Power-ups: a green star makes the basket wider for 5 seconds.



Two players: A/D keys move a second basket. Who catches more?



Next class link: control the basket with a joystick or potentiometer (YI-32!).

Think like an engineer

Why do we use clock.tick(60)? What happens on a very fast laptop without it?

Boss level

Add levels: every 10 points the background colour changes and a new obstacle appears.

Objective check

Can I really do it? Prove it.

01	Point to INPUT, UPDATE and DRAW in your code.	○ ○ ○
02	Show the line that detects a catch. What does colliderect do?	○ ○ ○
03	Show your lives, speed-up and game over working.	○ ○ ○

Rate yourself

Colour one circle each. Red: not yet. Amber: with help. Green: I can explain it to a friend.

Teacher: Arcade hour: everyone plays 3 classmates' games and leaves a 'star' comment.

Show what you know

What did we learn?

- [x] Games are a loop: input → update → draw
- [x] clock.tick(60) = 60 frames per second
- [x] Rects hold position and size; y grows down
- [x] colliderect() detects catches
- [x] Lives, speed-up and game over make it a game

Exit ticket: what would happen if we forgot screen.fill() at the start of DRAW?

At home

Teach a younger sibling to play your game and watch how they learn it.



Next class:
ESP32-CAM Wildlife Camera!